CASS Sustainability Metrics Report

The Center for Open-Source Research Software Stewardship and Advancement (CORSA)
Gregory R. Watson, Oak Ridge National Laboratory, Oak Ridge, TN, USA
Addi Malviya-Thakur, Oak Ridge National Laboratory, Oak Ridge, TN, USA
M. Scot Breitenfeld, The HDF Group, Champaign, IL, USA
Daniel S. Katz, National Center for Supercomputing Applications, Champaign, IL, USA
Elaine M. Raybourn, Sandia National Laboratories, Albuquerque, NM, USA
Bill Hoffman, Kitware, Clifton Park, NY, USA

Metrics Working Group
The authors are grateful for the Metrics Working Group's input, which significantly refined the discussion of various metrics. Their expertise proved invaluable.

Revision History

Version	Date	Description
1.0	2025.5.28	Initial draft created for review by the Metrics Working Group.
2.0	2025.10.1	Revised version incorporating feedback and insights from the Metrics Working Group review.

Acknowledgments
This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing under contract number DE-AC05-00OR22725.

1 Introduction
1.1 Document Overview and Organization
2 Summary of Metrics
2.1 Using the Metrics Summary Table
3 Implementation Recommendations
3.1 Getting Started (Months 1-3)
3.2 Building Momentum (Months 4-12)
3.3 Long-term Development (Year 2+)
3.4 Avoiding Common Pitfalls
4 Detailed Metric Descriptions
4.1 Impact
4.1.1 Software Citation and Adoption
Metric-rationale
Measurement Methods
Considerations
4.1.2 Field Research Impact
Metric-rationale
Measurement Methods
Considerations
4.2 Community
Community Overview - Common Measurement Infrastructure
4.2.1 Codes of Conduct (CoC), Governance, and Contributor Guidelines
Metric-rationale
Measurement Methods
Considerations
4.2.2 Open-Source Licensing and FAIR Compliance
Metric-rationale
Measurement Methods
Considerations
4.2.3 Active Maintenance
Metric-rationale
Measurement Methods
Considerations
4.2.4 Engagement
Metric-rationale
Measurement Methods
Considerations
4.2.5 Outreach
Metric-rationale

Measurement Methods
Considerations
4.2.6 Welcomeness
Metric-rationale
Measurement Methods
Considerations
4.2.7 Collaboration
Metric-rationale
Measurement Methods
Considerations
4.3 Viability
4.3.1 Financial Sustainability
Metric-rationale
Measurement Methods
Considerations
4.3.2 Institutional & Organizational Support
4Metric-rationale
Measurement Methods
Considerations
4.3.3 Project Longevity and Community Health
Metric-rationale
Measurement Methods
Considerations
4.4 Quality
4.4.1 Reliability and Robustness
Metric-rationale
Measurement Methods
Considerations
4.4.2 Development Practices
Metric-rationale
Measurement Methods
Considerations
4.4.3 Reproducibility
Metric-rationale
Measurement Methods
Considerations
4.4.4 Usability
Metric-rationale
Measurement Methods
Considerations
4.4.5 Accessibility

Metric-rationale
Measurement Methods
Considerations
4.4.6 Maintainability and Understandability
Metric-rationale
Measurement Methods
Considerations
4.4.7 Performance and Efficiency
Metric-rationale
Measurement Methods
Considerations
4.5 Conclusion and Strategic Recommendations
5 Emerging and Supplementary Metrics
5.1 Environmental Efficiency
Metric-rationale
Measurement Methods
Considerations
5.2 HPC Scalability and Accelerator Utilization
Metric-rationale
Measurement Methods
Interview Questions:
Considerations
6 Conclusion
7 Bibliography

1 Introduction

This guide outlines the sustainability metrics curated and refined by the Consortium for Advancement of Scientific Software (CASS), based on existing research in software engineering and software metrics. These metrics form a framework for considering the long-term viability of research software projects. Sustainability is a complex concept that encompasses various aspects, including a project's impact, the health of its community, its long-term viability and organizational robustness, as well as the quality of the software.

The metrics are categorized into four main dimensions: **Impact**, **Community**, **Viability**, and **Quality**. This document provides a comprehensive breakdown of the metrics within each dimension, including the rationale behind them, measurement methods, and key considerations that can help projects evaluate and enhance their sustainability. Projects should interpret these metrics as evidence-informed guidance rather than validated predictors of sustainability success. Metrics should be adapted to specific project contexts, organizational cultures, and domain requirements. In this framework, Sustainability metrics from the Impact Framework are reflected through the combination of Viability and Community metrics**.** *The Implementation Recommendations* section offers a phased approach to starting the sustainability assessment.

1.1 Document Overview and Organization

This report is designed to facilitate both quick reference and detailed implementation planning for sustainability efforts. It begins with a Summary of Metrics table, which provides an overview of all sustainability indicators categorized into the four core dimensions. Following this summary, you'll find comprehensive descriptions of each metric, detailing the rationale, measurement methodologies, interview questions, and key considerations for each category. The report concludes with strategic recommendations, emerging metrics, and a consolidated reference list.

For support during implementation, the Center for Open Source Research Software Stewardship and Advancement (CORSA) provides guidance and resources. CORSA can assist research software projects in navigating sustainability assessment and planning. To learn more, subscribe to the CORSA mailing list by signing up at https://corsa.center/ or reach out to info@corsa.center.

2 Summary of Metrics

2.1 Using the Metrics Summary Table

The following table provides an overview of sustainability indicators across four dimensions: Impact, Community, Viability, and Quality. Each row lists a metric category along with example

indicators that can be assessed through either automated analysis, manual assessments, or project interviews.

This table is designed to help project teams quickly identify relevant metric topics, enabling them to access sustainability evidence at a glance. The indicators are examples rather than exhaustive requirements. For details on measurement methodologies and implementation, and other considerations, refer to the corresponding sections of the report.

	METRIC CATEGORY	EXAMPLE INDICATORS
IMPACT	Citation	• Number of citations/mentions • Metadata: CITATION.cff and/or codemeta.json • Number of DOIs resolutions or downloads
	Adoption	• Number of dependent packages • Evidence of inclusion in training material or training activity • Number of Spack recipe instances
	Field Research Impact	• Number of non-bibliographic references to DOIs or metadata in publications and project documents • Recommendation or acknowledgment by a High-Performance Computing Center/Facility in official documentation or software stack listings • Community uptake signals from open platforms such as Forks/stars of the project
COMMUNITY	Codes of Conduct (CoC), Governance, and Contributor Guidelines	• Presence of CODE_OF_CONDUCT.md, CONTRIBUTING.md, and GOVERNANCE.md • Repository documentation (e.g., README, CONTRIBUTING, GOVERNANCE files) contains keywords about governance structures and decision-making practices
	Licensing	• Presence of license (GitHub API) or SPDX identifiers. • License is OSI-approved.
	Active Maintenance	• Commit gaps or archive notices indicating unmaintained code.
	Engagement	• Time to respond to and close support issues • Median time to first response on issues/PRs • Ratio of closed vs. opened support requests over time • Percentage of issues resolved by non-core members
	Outreach	• Number of new contributors per release cycle • Ratio of new vs repeat participants in community discussions • Percentage of new vs repeat contributors

		Welcomeness	• CHAOSS psychological safety metrics (inclusion, learner, contributor, challenger safety) • Presence of onboarding materials, mentorship programs • Evidence of representative leadership roles • Evidence of transparency in communication and decision-making
		Collaboration	• Evidence of participation in external events, mentions in cross-project forums or domain-wide mailing lists • Evidence of cross-project references or collaboration tags in issues/PRs • Evidence that software has been designed for interoperability (dependencies, plugins, API endpoints)
	VIABILITY	Financial and Funding Sustainability	• Number of funding sources at one time • Income from non-grant sources • Funders provide multi-year commitments or sustainability guarantees • Participation in specific funding programs designed to provide sustained support for research software
	VIABILITY	Institutional & Organizational Support	• Level of industry or institutional adoption • Organizations dedicate staff to contribute code, documentation, or maintenance directly to the project.
	VIABILITY	Project Longevity	• Number of active contributors over time • Frequency of releases and feature updates • Absence of `maintenance mode' indicators such as pinned issues or tags like “archived” or “no longer maintained”.
	QUALITY	Development Practices	• Use of CI/CD, testing, and linting • Evidence of peer code review practices • Use of issue trackers and community forums
	QUALITY	Reproducibility	• Indicators that the software and its intended usage workflows support reproducible results. • Indicators of containerization • FAIR4RS compliance scores
	QUALITY	Usability	• Presence of installation instructions, setup scripts, and onboarding guides • Presence of documentation, API references, tutorials, and user guides • Clear documentation for setup across different platforms and clusters • User Experience Questionnaire (UEQ) scores

		Portability	• Presence of installation scripts and portable build systems (e.g., Spack, Conda, CMake) • Availability in containers (Docker, Singularity) or pre-installed HPC facility environments • Availability in pre-installed environments (containers, vendor stack, E4S, user HPC facilities and centers) • Evidence of compatibility with accelerators and modern architectures
		Maintainability and Understandability	• Use of Static analysis of readability and complexity • Presence of developer documentation, coding standards, etc.
		Performance and Efficiency	• Indicators of static and runtime performance checking • Availability and performance on modern accelerators (GPUs) • Scalability on large systems • Software Carbon Intensity metrics
		Reliability and Robustness	• Results of static and runtime checking for bugs • Evidence of CERT Secure Coding Guidelines

3 Implementation Recommendations

Attempting to acquire metrics from all categories at once is likely to require a significant effort by projects. Instead, we suggest that projects begin with a set of foundational indicators that require minimal resources to implement, such as the presence of a governance document or a commit pattern analysis, and progress to more complex metrics, including *welcomeness*. One possible strategy for a staged implementation of sustainability metrics is outlined below.

3.1 Getting Started (Months 1-3)

Goal — To establish a baseline understanding of project health and implement foundational governance.

1. Establish baseline assessment using automated tools/dashboards for basic metrics
2. Implement essential governance documentation (CODE_OF_CONDUCT.md, CONTRIBUTING.md, LICENSE)
3. Set up basic citation tracking using available APIs and tools
4. Assess the current funding situation and document sustainability planning needs

3.2 Building Momentum (Months 4-12)

Goal — To deepen insights with targeted metrics and formalize sustainability planning.

1. Expand community health monitoring using CHAOSS tools where appropriate
2. Implement quality assurance practices appropriate to project size and context
3. Develop stakeholder engagement around sustainability planning

4. Adopt a sustainability dashboard for regular monitoring and reporting

3.3 Long-term Development (Year 2+)

Goal — To embed sustainability as a core strategic function of the project and to establish leadership by contributing knowledge back to the broader community.

1. Establish a comprehensive sustainability strategy based on metric insights
2. Implement advanced measurement approaches where warranted and cost-effective
3. Contribute to validation research by sharing experiences and outcomes
4. Mentor other projects in sustainability assessment and planning

3.4 Avoiding Common Pitfalls

1. Metric overload — Focus on metrics most relevant to your project context and goals, and prioritize metrics that reflect desired *outcomes*, not just raw output.
2. Automation bias — Remember that automated tools provide indicators, not definitive assessments.
3. Context ignorance — Adapt frameworks to your specific domain, culture, and organizational context.
4. Validation assumptions — Acknowledge uncertainty and use metrics as guidance rather than definitive measures.
5. Resource mismatch — Ensure metric implementation efforts align with available resources and priorities.
6. Stakeholder Skepticism — Metrics are ineffective if they are not understood, trusted, and utilized by the community and its leads. Ensure transparent communication about *why* specific metrics are tracked and *how* they will influence decisions.
7. Inaction on Insight — Failure to take action on insights means that data collection is futile (e.g., creating beautiful dashboards that no one uses to make decisions). Create a straightforward process for regularly reviewing metrics and converting findings into actionable steps or strategic adjustments.

4 Detailed Metric Descriptions

The following sections offer detailed guidance on implementing each sustainability metric across the four core dimensions. Each metric description follows a consistent format: a **rationale** explaining its importance for the long-term viability of the software, **methods for measurement** that include both automated repository analysis and qualitative interview questions, and **considerations** outlining contextual factors, potential pitfalls, and interpretation guidelines that practitioners should keep in mind during their assessment.

Throughout these sections, readers will encounter both technical measurement techniques and human-centered interview questions. This approach reflects the framework's belief that sustainable software requires not only quantitative indicators but also a qualitative understanding of community dynamics, institutional context, and stakeholder needs. The measurement methods emphasize automation wherever possible to lessen the assessment

burden. At the same time, the interview questions serve as guides for gathering insights that cannot be captured through repository analysis alone.

4.1 Impact

The ***Impact*** dimension assesses the influence of software based on its adoption, citation, and integration into research workflows. Direct indicators include citations in scientific literature and its use as a dependency in other packages, while indirect measures encompass inclusion in training materials and acknowledgments from HPC facilities and centers (e.g. https://docs.olcf.ornl.gov/software/index.html). High-impact software attracts resources and institutional support, creating feedback loops that enhance its viability.

4.1.1 Software Citation and Adoption

Software Citation and Adoption measures the formal recognition and widespread use of software within the research community through citations, dependencies, and educational materials.

Metric-rationale

Proper software citation is crucial for the sustainability of research software, as it ensures that developers receive academic recognition for their work and promotes ongoing investment in software maintenance and development. High adoption by other projects and inclusion in training materials are strong indicators of impact, demonstrating the software's value to the broader research community. Recent improvements in citation tracking include enhanced metadata standards and AI-powered citation extraction that captures informal mentions previously missed by traditional bibliometric tools.

Measurement Methods

Repository Mining Approach:

- **Enhanced Citations and Mentions**: Advanced tools, including Semantic Scholar's enhanced API (2024), OpenAlex's comprehensive scholarly database, and AI-powered citation extraction from preprints and grey literature
- **Improved DOI Tracking**: Integration with Software Heritage, Zenodo enhanced APIs, and emerging platforms like DataCite for comprehensive software publication tracking
- **Comprehensive Citation Metadata**: Enhanced CITATION.cff and codemeta.json detection with validation tools and automated metadata quality assessment. Both citation file formats provide machine-readable metadata for software citation, with codemeta.json following the CodeMeta standard for software metadata exchange

- **Advanced Dependency Analysis**: Multi-platform ecosystem mapping including Spack, conda-forge, PyPI, CRAN, Bioconductor, and domain-specific package managers
- **AI-Enhanced Training Detection**: Machine learning-powered analysis of educational content across platforms, including Coursera, edX, institutional repositories, and GitHub classroom materials

Interview Questions:

1. What research publications have cited or used your software? How is your software typically cited?
2. Do you provide a DOI for your releases, and do you track both formal citations and informal mentions?
3. Is your software used by other open-source projects? Can you name specific high-impact dependents?
4. Has your software been included in any academic courses, workshops, or training programs?
5. How do you track comprehensive mentions or usage of your software across academic literature, including preprints and grey literature?

Considerations

Citation tracking must account for varying practices across research domains and the potential lag between software use and formal citation. Enhanced AI-powered tools now capture informal mentions that traditional bibliometric analysis missed, providing a more comprehensive impact assessment. The presence of CITATION.cff files should be encouraged, but historical citation methods must also be captured. Zenodo integration may not be universal, requiring alternative DOI sources. Dependency analysis provides concrete usage evidence but may not capture the full scope of software influence, particularly in domains with limited adoption of package managers.

4.1.2 Field Research Impact

Field Research Impact measures the software's significant influence and essential role within its scientific community, demonstrating its contribution to research advancements and institutional adoption.

Metric-rationale

This metric captures the software's role in enabling scientific discoveries and its integration into major research infrastructure. High field research impact indicates deep embedding within scientific workflows and institutional recognition, both of which are crucial for attracting ongoing support and ensuring long-term sustainability. The value is demonstrated through contributions

to specific research advancements, adoption by major institutions, and the enabling of new scientific methodologies. Recent advances in AI-powered content analysis enable the detection of software impact in scientific literature with greater sophistication.

Measurement Methods

Repository Mining Approach:

- **AI-Enhanced Publication Analysis**: Large language model-powered analysis of scientific literature to identify software-enabled discoveries and methodological innovations
- **Comprehensive Institutional Tracking**: Advanced web scraping and API integration with major research facilities, national laboratories, and computational centers.,**Enhanced Metadata Linking**: Cross-referencing software releases with breakthrough research announcements using DOI networks and institutional press releases
- **Impact Narrative Extraction**: Natural language processing to identify and categorize impact claims from publications, facility reports, and research announcements

Interview Questions:

1. Can you describe specific scientific discoveries that were enabled by your software?
2. Which national labs, research institutes, or universities actively use your software in their research workflows?
3. Has your software supported new methodologies or research techniques in your field?
4. What features or capabilities have you developed to support emerging scientific techniques?
5. How do you distinguish between software installation and actual research usage in scientific applications?

Considerations

Field research impact is often challenging to quantify automatically and may require qualitative assessment through interviews. The distinction between software installation on systems and actual research usage is crucial but challenging to measure systematically. Impact may be indirect, with software contributing to the research infrastructure rather than directly producing scientific outputs. AI-powered analysis tools are improving, but may miss nuanced impact narratives that require domain expertise to interpret correctly.

4.2 Community

The ***Community*** dimension assesses the health and engagement of developers, maintainers, and users. Thriving communities offer redundancy in maintainership, diverse perspectives for problem-solving, and mechanisms for knowledge transfer, all of which are essential for

continuity. The evaluation focuses on governance structures, such as codes of conduct and contributor guidelines, as well as responsiveness to users, outreach to new contributors, and the promotion of psychological safety.

Community Overview - Common Measurement Infrastructure

The Community dimension metrics utilize a shared measurement infrastructure to ensure consistent data collection and avoid redundant tooling across individual metrics. This common foundation includes:

CHAOSS GrimoireLab Platform: The enhanced GrimoireLab platform provides the technical backbone for community analytics, featuring OpenSearch migration for improved performance, multi-repository analysis capabilities, and scalability enhancements. The platform integrates automated detection algorithms for psychological safety indicators and employs machine learning models trained on successful community patterns to provide predictive community health indicators.

Multi-Platform Data Integration: Comprehensive data collection spans multiple communication and collaboration platforms, including Git, GitHub, GitLab, Jira, Slack, Discord, Discourse, mailing lists, and other community communication channels. This integrated approach provides holistic visibility into community dynamics across fragmented communication ecosystems.

AI-Powered Analysis Tools: Advanced sentiment analysis, contribution pattern recognition, and conflict resolution tracking leverage natural language processing and machine learning to identify trends and patterns that manual analysis might miss.

Cross-Repository Network Analysis: Sophisticated network analysis identifies collaboration patterns and knowledge sharing across project boundaries, revealing hidden connections and collaboration opportunities within the broader ecosystem.

Individual metrics detailed below specify which aspects of this infrastructure they utilize and how data should be interpreted within specific metric contexts.

4.2.1 Codes of Conduct (CoC), Governance, and Contributor Guidelines

This metric evaluates the presence and effectiveness of formal governance structures, codes of conduct, and contributor guidelines that establish community norms and decision-making processes.

Metric-rationale

Research consistently demonstrates that clear governance structures, including Codes of Conduct (CoC) and contributor guidelines, significantly enhance the sustainability of open-source research software. These documents establish community norms, decision-making processes, and conflict resolution mechanisms that enable effective collaboration and inclusive participation. Recent developments include enhanced OpenSSF Best Practices Badge requirements that standardize governance documentation discovery and improved automated assessment tools.

Measurement Methods

Repository Mining Approach:

- **Enhanced Document Detection**: Advanced file scanning for `CODE_OF_CONDUCT.md`, `CONTRIBUTING.md`, `GOVERNANCE.md`, and variant naming conventions using GitHub Contents API and enhanced pattern matching
- **Governance Keyword Analysis**: Natural language processing to detect governance-related keywords, decision-making processes, and community structure indicators
- **OpenSSF Badge Integration**: Automated assessment of OpenSSF Best Practices Badge completion status, particularly governance documentation requirements
- **CHAOSS Governance Metrics**: Implementation of standardized CHAOSS governance health indicators, including decision-making transparency and community participation metrics
- **Governance Effectiveness Assessment**: Analysis of issue resolution patterns, decision implementation tracking, and community participation in governance processes

Interview Questions:

1. Do you have published codes of conduct, governance documents, and contributor guidelines?
2. How are major decisions made in your project (e.g., BDFL, consensus, voting, committee-based)?
3. What processes exist for handling conflicts, disputes, or violations of community standards?
4. How do you onboard new contributors and ensure they understand project governance?
5. What mechanisms ensure transparency and accountability in project leadership and decision-making?

Considerations

While `CODE_OF_CONDUCT.md` and `CONTRIBUTING.md` are increasingly standardized and discoverable, governance documents lack consistent naming conventions, making automated discovery challenging. The OpenSSF Best Practices Badge program addresses this by

requiring governance documentation URLs, creating standardized, machine-readable data sources. Document presence is necessary but insufficient—effectiveness requires qualitative assessment of implementation and community satisfaction. Recent research emphasizes the importance of psychological safety in governance effectiveness, requiring assessment beyond document existence.

4.2.2 Open-Source Licensing and FAIR Compliance

This metric assesses the clarity and appropriateness of software licensing, with a particular focus on OSI-approved licenses that support community participation, and evaluates compliance with the FAIR for Research Software (FAIR4RS) principles.

Metric-rationale

Open-source licensing plays a crucial role in ensuring the sustainability of research software by facilitating accessibility, legal reuse, and continued maintenance. Licenses such as MIT, Apache, and GPL support broad community participation and foster collaborative development while providing legal clarity for users and contributors. The FAIR4RS principles, officially released in May 2022, have gained rapid institutional adoption and provide standardized criteria for research software findability, accessibility, interoperability, and reusability.

Measurement Methods

Repository Mining Approach:

- **Enhanced License Detection**: Advanced SPDX license identification tools with improved parsing of license exceptions (e.g., "Apache 2.0 with LLVM exceptions")
- **Automated FAIR4RS Assessment**: Integration with FAIR-IMPACT project tools, including FAIRsoft and F-UJI Extended Service for quantitative compliance checking
- **OSI License Validation**: Comprehensive comparison against the Open Source Initiative-approved license list with automated compliance scoring
- **License Exception Handling**: Specialized tools for detecting and correctly categorizing license modifications and exceptions that standard APIs miss
- **FAIR Metadata Assessment**: Automated evaluation of research software metadata quality, documentation completeness, and interoperability standards

Interview Questions:

1. What license does your software use, and how did you select it for your specific research context?
2. How do you ensure your software remains legally reusable across different institutional and international contexts?

3. Have you assessed your software's compliance with FAIR4RS principles?
4. What challenges have you encountered regarding licensing or FAIR compliance?
5. How do you handle contributions from external developers regarding licensing and compliance requirements?

Considerations

Automated tools often fail to correctly parse licenses with exceptions, requiring specialized SPDX license matching tools and explicit SPDX identifiers within license files. OSI-approved licenses generally provide better community support and legal clarity; however, the specific license choice should align with project goals and institutional requirements. The FAIR4RS assessment is increasingly automated, but it still requires domain expertise for meaningful interpretation. Compliance requirements may vary significantly across institutions and funding agencies, requiring flexible assessment approaches.

4.2.3 Active Maintenance

This metric measures project activity levels, maintenance status, and indicators of project abandonment or transition to maintenance-only mode.

Metric-rationale

Software activity and maintenance status are essential indicators of project sustainability and health. Active maintenance reflects an ongoing commitment to the software and its user community. Conversely, gaps in activity or explicit maintenance notices help users and stakeholders understand the project's status and plan accordingly. Research shows that 16% of open source projects by major tech companies become inactive within 12 months, and commit gaps exceeding 12 months strongly indicate project abandonment. Understanding maintenance patterns helps differentiate between healthy, mature projects with slower release cycles and those at risk of abandonment. The abandonment of active developers poses significant sustainability risks, as reliance on a few contributors creates vulnerability when key personnel leave.

Measurement Methods

Repository Mining Approach:

- **Commit Activity Pattern Analysis**: Evaluation of commit frequency, seasonal trends, and gaps in development activity using 20-day rolling windows to forecast contributor activity

- **Maintenance Mode Indicator Detection**: Automated identification of archive status flags, maintenance mode labels, "seeking maintainer" notices, and explicit abandonment announcements in README files or repository descriptions
- **Activity Trend Monitoring**: Tracking of overall project activity patterns to distinguish between natural periods of lower activity (stable, mature projects) and concerning declines that may indicate abandonment risk. This differs from engagement response time metrics by focusing on aggregate activity levels rather than interaction quality
- **Release Pattern Assessment**: Examination of release frequency, versioning schemes, semantic versioning compliance, and the ratio of feature development to bug fixes over time
- **Multi-Channel Communication Activity**: Observation of mailing lists, forums, Slack/Discord channels, and other community communication platforms for signs of active engagement or silence, focusing on presence/absence of activity rather than response quality
- **Contributor Abandonment Forecasting**: Probabilistic modeling based on recent contribution patterns to identify contributors at risk of abandonment using survival analysis techniques

Interview Questions:

1. What is your current maintenance commitment level and expected timeline for active development versus maintenance-only support?
2. How do you communicate project status changes, transitions to maintenance mode, or plans for seeking new maintainers to your user community?
3. What specific circumstances or triggers would lead you to formally declare the project as entering maintenance mode or seeking additional maintainers?
4. How do you prioritize maintenance activities when development resources become limited or contributor availability decreases?
5. What succession planning mechanisms exist for continued maintenance if current maintainers become unavailable due to career changes, funding loss, or other circumstances?

Considerations

Mature, stable projects may show lower commit activity without indicating issues, as reduced activity often reflects stability rather than decline. Academic software patterns vary with funding cycles, usually resulting in gaps during summers or between grant periods. Maintenance mode applies to projects that have met their goals and only require bug fixes and security updates.

Distinguishing temporary inactivity from abandonment depends on project goals, funding, and contributors. Research shows that contributors with breaks of under a year have a 35-55% chance of returning, but this drops to 21-26% for breaks of over a year. Projects with few maintainers are more at risk of abandonment due to the "bus factor.

Cultural factors influence the interpretation of activity – some communities expect slower updates, while industry projects may demand more frequent ones. Archive status should be interpreted carefully, as some projects are intentionally archived to indicate completion.

4.2.4 Engagement

Engagement evaluates the responsiveness and interaction quality between project maintainers and the community, measuring how effectively projects address user needs, support requests, and community participation through timely responses and sustained interaction patterns.

Metric-rationale

Mozilla research shows that maintainer responsiveness encourages repeat contributions. Open source projects are dynamic and influenced by social behavior, making engagement assessment vital. Issue response time indicates project responsiveness and community health. Quick responses foster active involvement, while delays can discourage contributors. Fast feedback keeps contributors engaged, promoting rapid iteration. Engagement metrics like issue resolution, pull request merges, and activity patterns gauge repository stability and recovery. The quality of engagement—response helpfulness and follow-through—is often as important as speed.

Measurement Methods

Repository Mining Approach:

- **Response Time Tracking**: Measurement of time to first response for issues, pull requests, and discussions, signaling community activity and engagement. Analysis filters responses by role (maintainer vs. contributor), excludes automated bot responses, and tracks based on whether the issue opener is a new or long-time participant
- **Issue Resolution Analysis**: Tracking of issue resolution time, the ratio of open to closed issues, and identification of persistent backlogs that may indicate maintainer overload or declining user experience
- **Pull Request Flow Assessment**: Analysis of pull request opening and closing patterns, time to merge, and the ratio of PRs accepted versus closed without merge. Projects should consistently close PRs without falling behind, with responses to PRs ideally occurring within two business days
- **Engagement Quality Metrics**: Analysis of interaction patterns focusing on the depth and quality of maintainer responses, including comprehensiveness of feedback, actionability of suggestions, and follow-through on commitments. Assessment of whether responses provide clear next steps and maintain a constructive tone
- **Communication Pattern Analysis**: Examination of response consistency across different issue types (bugs, features, support questions), contributor types (new, casual,

core), and communication channels to identify potential gaps or biases in engagement patterns
- **Support Request Closure Analysis**: Tracking the ratio of closed versus opened support requests over time to identify trends in community responsiveness and maintainer capacity. Sustained negative ratios (more open than closed) indicate growing backlog concerns
- **Community Participation Assessment**: Analysis of issue resolution patterns, including the percentage of issues resolved by non-core members, indicating distributed decision-making and community empowerment beyond core maintainer teams

Interview Questions:

1. How do you prioritize and manage incoming support requests, bug reports, and community questions?
2. What is your target response time for different types of community interactions (critical bugs, feature requests, general questions)?
3. How do you maintain engagement quality during periods of high activity or limited maintainer availability?
4. What strategies help you balance responsive engagement with other development priorities and maintainer wellbeing?
5. How do you measure community satisfaction with support quality and responsiveness?

Considerations

A lengthy response delay may signal low community engagement, while quicker responses can boost participation. Response times should consider volunteer availability, time zones, and academic schedules, where maintainers often have competing priorities. Responding doesn't always mean immediate action—acknowledgment and an estimated timeline can suffice.

Different issue types require different response expectations. Backlogs can indicate a need for more maintainers or poor user experience, while low PR acceptance might reflect overwhelmed maintainers or overly critical reviewing, indicating a need for guidance.

Static metrics offer insights but may miss the dynamic aspects of open source health. Declining engagement often begins months before a developer's departure, serving as an early warning signal.

High engagement demands can be unsustainable, so developing community self-help and distributed support is crucial. Projects should aim for timely human responses, as unresponsive maintainers cause bottlenecks. Realistic expectations help balance community needs with maintainer sustainability, preventing burnout.

4.2.5 Outreach

Outreach measures the project's ability to attract, engage, and integrate new contributors while expanding community participation through external events, training activities, and strategic community-building efforts.

Metric-rationale

Attracting and retaining contributors is vital for open-source projects' survival. Without new contributors, communities become stagnant as active members move on. Ongoing influx of newcomers sustains vitality and longevity.

Research shows community-based open source projects depend on continuous newcomer flow for survival. A healthy project attracts newcomers; declines suggest engagement issues. Projects with 25% 'Good First Issues' see 13% more contributors; those with 40% see 21% more.

Tracking first-time contributors helps gauge recruitment success and retention. Contributing to open source is key for developers, and effective onboarding boosts sustainability. Contributions include coding, community management, bug triage, evangelism, user support, and more, reflecting project maturity activity.

Measurement Methods

Repository Mining Approach:

- **New Contributor Tracking**: Measurement of the number of new contributors making their first contribution to a project across key community interactions, including code commits, issue creation, pull requests, and code reviews. Analysis tracks new contributors by time period to identify trends and changes in project engagement, providing leading indicators of community growth or decline
- **Contributor Retention Analysis**: Comprehensive tracking of contributor progression from first-time to casual to repeat contributors. First-time contributors are those making their initial contribution; casual contributors may be defined as those with a low number of commits (one commit, fewer than five commits, or project-specific thresholds); repeat contributors demonstrate sustained engagement over multiple contribution cycles. Analysis includes cohort analysis to understand what percentage of new contributors return for second, third, and subsequent contributions, and identifies critical drop-off points in the contributor journey
- **Contributor Lifecycle Mapping**: Longitudinal analysis tracking individual contributor trajectories from initial discovery through sustained participation, identifying factors that predict successful transition from one-time to regular contributors. Measurement of the time intervals between contributions and patterns that precede contributor departure or increased engagement

- **Contribution Type Diversity**: Assessment of the variety of contribution types beyond code, including documentation, community management, event participation, mentorship, translation, design work, and other non-code contributions. A variety of contribution types demonstrates project maturity and provides multiple pathways for community participation, reducing barriers to entry and increasing accessibility for diverse skill sets
- **Good First Issue Effectiveness**: Analysis of issues labeled for newcomers (e.g., "good first issue," "newcomer," "help wanted") and their impact on new contributor attraction. Monitoring includes the frequency of issue creation, resolution rates, time-to-claim metrics, and correlation between good first issue availability and new contributor onboarding success. Assessment of whether these issues lead to subsequent contributions beyond the initial engagement
- **External Event Participation**: Tracking of project representation at conferences, workshops, meetups, and training events. Evidence includes speaker participation at domain-specific and general software engineering conferences, booth presence at community events, workshop facilitation, conference sponsorship, and organization of project-specific gatherings. Correlation analysis between event participation and spikes in new contributor activity
- **Training Material Integration**: Detection of project inclusion in educational content across platforms, including Coursera, edX, institutional repositories, GitHub Classroom materials, university curricula, professional training programs, and bootcamp curricula. Assessment includes tracking mentions in course syllabi, tutorial creation by community members, and integration into established training pathways for the project's domain
- **Onboarding Infrastructure Assessment**: Evaluation of onboarding resources, including getting-started guides, contribution pathways for different skill levels and interests, mentorship program structure and activity, and documentation specifically targeting new contributors. Analysis of onboarding issue template usage and effectiveness of automated onboarding workflows

Interview Questions:

1. What strategies does your project use to attract new contributors, and how do you measure their effectiveness?
2. How do you onboard new contributors, and what resources are available to help them get started?
3. What percentage of your contributors are first-time versus repeat contributors, and how has this changed over time?
4. What is your contributor retention rate (percentage of first-time contributors who return for additional contributions), and how does this compare to similar projects?
5. Does your project participate in external events such as conferences, workshops, or training programs? How does this impact community growth?
6. What barriers do new contributors face when joining your project, and how do you address these challenges?
7. How do you distinguish between contributors who engage once versus those who become sustained community members, and what factors predict this transition?

Considerations

Contributor sustainability depends on recruiting and retaining contributors in open source projects, which face a shortage of helpers. Projects may fail if key contributors leave without replacements. Defining "contributor" broadly helps gauge community engagement beyond just merged code. Research shows that onboarding recommendations have mixed effects; practices should be tailored to specific contexts and experience levels. Documentation, especially contribution guidelines and README files, significantly boosts productivity and inclusivity. Outreach efforts are hard to measure quantitatively and may have delayed impacts, influenced by cultural and geographic factors. Balancing outreach with other priorities is essential to avoid burnout and ensure long-term community growth.

4.2.6 Welcomeness

This metric aims to evaluate factors related to community health and governance, explicitly focusing on contributors' experiences and community growth.

Metric Rationale

This metric evaluates the community environment and practices that impact contributor experience, retention, and project viability. Key areas of focus include the effective integration of new contributors, the overall quality of community interactions, and activities that promote software adoption and community growth.

Creating a welcoming and supportive environment is fundamental to community sustainability. Strong community engagement and positive contributor experiences lead to higher retention rates, facilitate broader software adoption, and enhance the long-term viability of the project. A welcoming community is also more likely to attract a diverse range of contributors, which can foster innovation and help reduce maintainer fatigue.

Measurement Methods

Repository Mining and Community Data Analysis:

- **CHAOSS Community Experience Metrics (Adaptation)**: Assessments based on the CHAOSS framework, including measures related to welcoming, learning, contributing, and proposing changes, often using automated data analysis.
- **Response Quality and Tone Analysis**: Evaluating the quality of communication in community support and engagement, considering not just response times but also tone, clarity, acknowledgment of contributions, and whether interactions are constructive or discouraging.
- **Communication Sentiment Analysis**: Automated tracking of sentiment in community channels to identify the general tone, potential areas of conflict, and changes in interaction quality over time.

- **Contributor Journey Mapping**: Automated analysis of onboarding success rates, retention patterns, progression across different contributor roles, and identification of common disengagement points or significant barriers faced by contributors.
- L**anguage and Communication Review**: Analyzing documentation and community communications for clarity, adherence to communication standards, and the use of welcoming and constructive language.
- **Leadership Role Representation**: Analyzing the distribution of leadership roles among different contributor demographics, geographical locations, and organizational affiliations to gain insights into the breadth of the governance structure.
- **Decision-Making Visibility**: Evaluating the accessibility of decision-making processes, documenting meeting summaries, roadmap transparency, and the extent of community input in significant project decisions.

Interview Questions

1. What processes are in place to ensure a consistently welcoming and supportive environment for all community members?
2. What specific practices promote a welcoming and accessible environment for a diverse array of contributors?
3. How are community questions and support requests managed to maintain a positive and constructive level of engagement?
4. What structured onboarding, documentation, or mentorship programs are available for new contributors?
5. How does the project track and address community satisfaction and concerns regarding contributor experience?

Considerations

Evaluating community experience factors requires careful implementation to ensure the analysis yields actionable insights without raising concerns about excessive oversight. The fragmented nature of communication platforms (e.g., GitHub, forums, chat, mailing lists) makes analyzing communication data challenging, particularly regarding private channels. Response time metrics should be viewed within the context of time zone differences, the availability of volunteer contributors, and diverse communication norms. Additionally, the quality and constructiveness of engagement are often more indicative of community health than response speed alone, necessitating a nuanced analysis of interaction patterns.

4.2.7 Collaboration

This metric assesses interoperability, cross-project collaborations, and integration with the broader scientific software ecosystem.

Metric-rationale

Research software that integrates well with widely used scientific tools demonstrates higher adoption rates and longer sustainability. Standardized APIs and common data exchange formats significantly improve long-term reuse. Software projects involved in cross-project collaborations tend to have more resilient governance and stronger developer communities. The Research Software Alliance's emphasis on ecosystem thinking has highlighted the importance of interoperability and collaboration in sustainable software development.

Measurement Methods

Repository Mining Approach:

- **Advanced Dependency Analysis**: Multi-platform ecosystem mapping using enhanced Spack integration, conda-forge analysis, and comprehensive package manager APIs
- **Cross-project Reference Detection**: AI-powered analysis of GitHub Issues and PRs for collaboration mentions, cross-project references, and shared development activities
- **Interoperability Assessment**: Automated analysis of API standards compliance, data format standardization, and integration capabilities
- **Collaboration Network Analysis**: Network analysis of contributor overlaps, shared dependencies, and cross-project communication patterns
- **Standards Compliance Tracking**: Assessment of adherence to domain-specific standards and protocols for scientific software interoperability

Interview Questions:

1. How does your software integrate with other tools in your scientific domain's ecosystem?
2. What collaborations exist with other open-source projects, and how are they maintained?
3. How do you ensure your software can integrate seamlessly with existing scientific workflows?
4. What standards or protocols does your software support for data exchange and interoperability?
5. How do you participate in broader ecosystem development through training events, workshops, or collaborative development?

Considerations

Assessing collaboration importance or criticality solely based on dependency existence remains challenging, as projects might have viable alternatives. Cross-project collaboration tracking through PR references is unreliable due to inconsistent documentation practices. Basic dependency information can be gathered automatically, but understanding collaboration depth and ecosystem integration requires qualitative assessment. Interoperability standards vary significantly across scientific domains, requiring domain-specific evaluation criteria.

4.3 Viability

The ***Viability*** dimension focuses on structural resilience, which refers to the ability to endure changes in funding, staff departures, and external pressures. Important factors include having diverse financial support, such as multiple funding sources and NIH Research Software Engineer awards. Institutional support is also crucial, particularly through dedicated Research Software Engineer positions. Additionally, distributing knowledge helps reduce risks associated with key individuals, and having evidence of continuity planning is vital for long-term sustainability.

4.3.1 Financial Sustainability

Financial Sustainability measures the diversity and stability of funding sources supporting the project, including grants, industry support, service revenue, and emerging institutional funding mechanisms.

Metric-rationale

Projects with well-defined long-term sustainability plans, including financial strategies beyond their initial grants, demonstrate significantly higher long-term viability. Diverse funding sources reduce dependency risk and indicate broad stakeholder value. Recent developments include NIH Research Software Engineer awards, providing dedicated RSE career support and increased institutional recognition of software maintenance funding needs. Some funding agencies now require explicit sustainability plans as part of grant applications.

Measurement Methods

Repository Mining Approach:

- **Enhanced Funding Documentation Analysis**: AI-powered parsing of `README.md`, `FUNDING.yml`, funding.json, and documentation for comprehensive sponsorship and funding acknowledgment detection
- **Institutional Affiliation Tracking**: Advanced analysis of contributor organizational diversity using GitHub Users API, LinkedIn integration, and institutional email domain analysis
- **NIH R50 Award Tracking**: Specific monitoring for NIH Research Software Engineer award recipients and similar dedicated funding mechanisms
- **Corporate Sponsorship Detection**: Automated detection of corporate funding, in-kind contributions, and industry partnership indicators
- **Funding Portfolio Analysis**: Comprehensive assessment of funding source diversity, amounts, and temporal distribution patterns

Interview Questions:

1. What is your current funding portfolio, and how do you plan for sustainability after major grants end?
2. What contingency strategies exist if primary funding sources are lost or reduced?
3. How do you balance research funding with infrastructure and maintenance funding needs?
4. Are contributors supported by dedicated RSE positions or institutional software development funding?
5. What services, partnerships, or alternative funding models support your software financially?

Considerations

Multiple funding sources reduce risk but don't guarantee adequate total funding for comprehensive project needs. Revenue and funding figures are often confidential and difficult to obtain through automated analysis. Contributor company affiliations may not reflect actual project support or institutional commitment. Some research software projects lack viable commercial models, requiring alternative sustainability approaches through institutional support. Funding diversity should be balanced against administrative overhead and management complexity.

4.3.2 Institutional & Organizational Support

This metric evaluates institutional support for research software maintenance and the establishment of professional career paths for Research Software Engineers (RSEs).

Metric-rationale

Establishing professional career paths for research software engineers (RSEs) is increasingly recognized as crucial for improving software sustainability. Universities and research projects that create dedicated RSE positions demonstrate significantly better long-term software viability. The 2024 NIH Research Software Engineer awards represent the first federal funding mechanism specifically for RSE career support, providing up to $300,000 over 3 years and indicating institutional recognition of RSE career pathway importance.

Measurement Methods

Repository Mining Approach:

- **RSE Position Detection**: Automated identification of contributors with dedicated RSE titles, software development roles, or research software career positions using LinkedIn API and institutional directory analysis

- **Institutional Support Tracking**: Analysis of funding acknowledgments for institutional software development commitments, dedicated development time, and career pathway indicators
- **Career Development Indicators**: Detection of professional development activities, RSE community participation, and career advancement opportunities
- **NIH R50 Award Integration**: Specific tracking of NIH Research Software Engineer award recipients and similar dedicated career funding mechanisms
- **Institutional Policy Analysis**: Assessment of institutional policies supporting research software development and RSE career recognition

Interview Questions:

1. Do team members have dedicated RSE positions, software development roles, or career development support?
2. What institutional support exists for long-term software maintenance and RSE career advancement?
3. How do you advocate for recognition of software development contributions within your institutional context?
4. What professional development opportunities exist for software developers in your organization?
5. How do institutional policies support research software sustainability and RSE career pathways?

Considerations

RSE positions and recognition vary significantly across institutions, countries, and research domains. Career path availability often depends on organizational culture and funding structures beyond individual project control. Institutional support may be informal and difficult to detect through automated means alone. The RSE movement is evolving rapidly, with varying levels of adoption across different research institutions and geographic regions.

4.3.3 Project Longevity and Community Health

This metric evaluates the project's long-term viability based on contributor diversity, consistent maintenance activity, the absence of abandonment indicators, and community viability factors.

Metric-rationale

Software projects that remain actively developed after their initial funding phase tend to have greater long-term scientific impact. A project with a single, overworked maintainer is significantly more fragile than one with a broad base of active contributors. Many funding evaluations now consider whether a project will continue to support existing features or transition into a

maintenance mode. Recent research emphasizes that individual metrics are insufficient for sustainability assessment without contextual understanding of project lifecycle stages.

Measurement Methods

Repository Mining Approach:

- **Comprehensive Activity Analysis**: Multi-dimensional tracking of commit history, release frequency, issue resolution patterns, and community engagement indicators
- **Contributor Viability Assessment**: Analysis of contributor diversity, retention patterns, knowledge distribution, and succession planning indicators
- **Maintenance Mode Detection**: Automated detection of sustainability warning indicators, including pinned maintenance notices, archive status, and reduced activity patterns
- **Community Health Trends**: Longitudinal analysis of community engagement patterns, contributor onboarding success, and long-term participation trends
- **Project Lifecycle Assessment**: Classification of project maturity stages with appropriate sustainability indicators for each lifecycle phase

Interview Questions:

1. How long has your project been in active development, and what are your long-term maintenance plans?
2. What succession planning exists for key contributors and knowledge transfer?
3. How do you distinguish between active development and maintenance phases in your project lifecycle?
4. What strategies help maintain contributor engagement during periods of reduced funding or activity?
5. How do you handle knowledge preservation and transfer when team members transition out of the project?

Considerations

"Active contributor" definitions must be consistent and meaningful across different project types and sizes. Contributor departure may be unrelated to project health, such as job changes, career transitions, or life circumstances. Mature, stable projects may naturally have lower activity levels without indicating sustainability problems. Release frequency should be evaluated in the context of project maturity, domain requirements, and user needs rather than arbitrary temporal targets.

4.4 Quality

The ***Quality*** dimension includes both technical excellence and user-facing features that reduce maintenance burdens while enhancing user confidence and participation. The assessment focuses on several areas:

- Reliability (including security practices and testing)
- Development practices (such as CI/CD and code reviews)
- Reproducibility (through containerization and FAIR4RS compliance)
- Usability (covering documentation and accessibility)
- Maintainability (focusing on code complexity and architecture)

4.4.1 Reliability and Robustness

Reliability and Robustness measure the software's ability to operate correctly under various conditions through static analysis, security practices, and adherence to coding standards.

Metric-rationale

Reliable software requires less corrective maintenance and builds greater trust among users, directly supporting long-term sustainability. High-quality software that follows security guidelines and passes static analysis checks demonstrates professional development practices and reduces the likelihood of critical failures that could damage the project's reputation. Modern AI-powered analysis tools provide significantly improved accuracy and actionable insights compared to traditional approaches.

Measurement Methods

Repository Mining Approach:

- **Advanced Static Analysis**: Modern tools like DeepSource, CodeAnt.ai, or SemGrep with AI-powered analysis, achieving improved false positive rates compared to traditional tools
- **Enhanced Security Analysis**: Comprehensive vulnerability detection using SAST/DAST tools, dependency vulnerability scanning, and automated security patch recommendations
- **CERT Guidelines Compliance**: Automated assessment of CERT Secure Coding Guidelines adherence with a specific focus on memory safety, input validation, and error handling
- **Test Coverage Excellence**: Advanced coverage analysis, including branch coverage, mutation testing, and coverage quality assessment beyond simple percentage metrics
- **Reliability Trend Analysis**: Longitudinal tracking of defect density, MTBF (Mean Time Between Failures), and reliability improvement patterns

Interview Questions:

1. What practices does your team follow to ensure software reliability and minimize critical failures?
2. How do you handle security considerations, vulnerability management, and secure coding practices?
3. What testing strategies help prevent bugs, regressions, and reliability issues?
4. How do you track and address code quality issues, technical debt, and reliability concerns over time?
5. What role do automated tools play in maintaining software robustness and security?

Considerations

Modern automated static analysis tools demonstrate significant improvements over traditional approaches, with substantially reduced false positive rates and more actionable insights. Security practices should be appropriate for the software's risk profile, user base, and deployment context. Coverage metrics should focus on quality rather than quantity, with a particular emphasis on critical path testing and edge case coverage. Warning density (warnings per KLOC) provides more meaningful comparisons than absolute counts across different project sizes.

4.4.2 Development Practices

This metric evaluates the adoption of modern development practices, including CI/CD, testing, code review, version control, and community engagement mechanisms.

Metric-rationale

Adopting technical best practices, such as version control, CI/CD, automated testing, and package managers, demonstrates professional development approaches that support long-term sustainability. These practices enhance code quality, minimize integration issues, facilitate community contributions, and lay a foundation for scalable development processes. Recent advances in development tooling and automation enable more sophisticated assessment of practice effectiveness.

Measurement Methods

Repository Mining Approach:

- **CI/CD Effectiveness Assessment**: Analysis of pipeline success rates, build performance, deployment frequency, and automation quality using GitHub Actions, GitLab CI, or other platform APIs
- **Testing Framework Excellence**: Comprehensive evaluation of testing practices, including unit testing, integration testing, performance testing, and security testing coverage
- **Code Review Quality Analysis**: Assessment of review participation, review thoroughness, defect detection rates, and knowledge sharing through review processes
- **Development Tool Integration**: Analysis of linter configuration, code formatting consistency, dependency management, and development environment standardization
- **Community Contribution Facilitation**: Evaluation of how development practices support external contributions and community engagement

Interview Questions:

1. What technical practices do you follow for CI/CD, automated testing, and quality assurance?
2. How do you manage code reviews, contributions, and release processes?
3. What development tools and workflows support your team's productivity and code quality?
4. How do you ensure consistent development practices across contributors and maintain standards over time?
5. How do your development practices facilitate community contributions and project growth?

Considerations

The presence of configuration files doesn't guarantee effective implementation—CI systems might only run trivial checks without meaningful quality gates. CI effectiveness should be measured through success rates, build performance, and actual defect prevention rather than just configuration presence. Different development practices may be appropriate for different project types, team sizes, and maturity levels. Practice adoption should support, rather than hinder, development, productivity, and community participation.

4.4.3 Reproducibility

Reproducibility measures the software's ability to generate consistent, verifiable results through the use of containerization, version control, documentation practices, and adherence to FAIR4RS principles.

Metric-rationale

Software sustainability is closely linked to whether computational results can be reproduced consistently across different environments and time periods. Best practices, such as version control, automated testing, containerization, and FAIR4RS compliance, improve both software reproducibility and longevity. Reproducible software builds greater trust in research outcomes, facilitates scientific validation, and supports reuse across different research contexts.

Measurement Methods

Repository Mining Approach:

- **FAIR4RS Compliance Assessment**: Automated evaluation using FAIR-IMPACT project tools, including FAIRsoft and F-UJI Extended Service for quantitative compliance scoring
- **Containerization Excellence**: Analysis of Docker, Singularity, or other container configurations with emphasis on build success, documentation quality, and reproducibility testing
- **Version Control Best Practices**: Assessment of semantic versioning, release management, dependency pinning, and reproducible build configurations
- **Environment Management**: Evaluation of dependency management practices, environment specification completeness, and cross-platform compatibility
- **Reproducibility Documentation**: Analysis of installation instructions, usage examples, and reproducibility claims with validation testing where possible

Interview Questions:

1. How do you ensure your software produces reproducible results across different computing environments?
2. What containerization, environment management, or dependency management practices do you use?
3. How do you handle version compatibility, dependency conflicts, and reproducibility across different platforms?
4. What documentation and examples support reproducible use of your software by other researchers?
5. How do you test and validate reproducibility claims across different systems and environments?

Considerations

Container presence doesn't guarantee functionality—containers must actually build and run correctly across different environments. Reproducibility requirements vary across scientific domains, with some fields requiring bit-for-bit reproducibility while others accept statistical reproducibility. The FAIR4RS assessment is increasingly automated, but it still requires domain expertise for meaningful interpretation. Documentation quality is crucial for reproducibility, but it is difficult to assess automatically without manual validation.

4.4.4 Usability

This metric evaluates ease of installation, documentation quality, user experience, and accessibility features that support diverse user needs and broad adoption.

Metric-rationale

Research software with clear, structured documentation and an excellent user experience is significantly more likely to be adopted and maintained in the long term. Projects that provide detailed API references, onboarding guides, and tutorial materials tend to see higher engagement and community contributions. Accessibility improvements ensure software can be used by diverse research communities, expanding impact and sustainability through broader adoption.

Measurement Methods

Repository Mining Approach:

- **User Experience Assessment**: Integration with validated instruments like the User Experience Questionnaire (UEQ) with psychometric validation across 30+ languages and established benchmarking
- **Documentation Completeness Analysis**: Automated evaluation of installation instructions, API documentation, tutorial availability, and user guide quality
- **Accessibility Feature Detection**: Analysis of internationalization support, UI accessibility features, alternative text for images, and inclusive design practices
- **Installation Success Tracking**: Automated testing of installation procedures across different platforms, package managers, and computing environments
- **Usage Analytics Integration**: Assessment of user engagement patterns, common user pathways, and usability pain points through analytics, where available

Interview Questions:

1. How easy is it for new users to install, configure, and begin using your software productively?
2. What accessibility improvements have you implemented to support the diverse needs and capabilities of your users?
3. How do you gather and respond to user experience feedback and usability concerns?
4. What documentation, tutorials, and onboarding materials support users with varying skill levels and different use cases?
5. How do you test and validate usability across different platforms, environments, and user contexts?

Considerations

The existence of documentation doesn't guarantee quality—a comprehensive manual review is often needed to assess clarity, completeness, and usability. Installation difficulty varies significantly across platforms, computing environments, and user technical backgrounds. Accessibility features may be domain-specific and difficult to assess automatically without specialized expertise. User experience improvements should be prioritized based on actual user feedback, usage analytics, and adoption patterns rather than theoretical considerations.

4.4.5 Accessibility

This metric evaluates the availability of portable build systems, container availability, architectural compatibility, and documentation that support diverse deployment environments.

Metric-rationale

Software Portability across different computing environments is crucial for broad adoption and long-term sustainability. Research computing environments vary dramatically in terms of available software, hardware architectures, and deployment constraints. Accessible software reduces barriers to adoption and enables use across diverse research communities and institutional contexts.

Measurement Methods

Repository Mining Approach:

- **Portable Build System Detection**: Analysis of installation scripts and portable build systems (e.g., Spack, Conda, CMake) with cross-platform compatibility assessment
- **Container Availability Assessment**: Evaluation of Docker, Singularity container availability and functionality across different container platforms
- **Architecture Compatibility Analysis**: Assessment of compatibility with accelerators, modern architectures, and diverse hardware configurations
- **Platform Documentation Evaluation**: Analysis of setup documentation quality across different platforms, clusters, and computing environments
- **Deployment Environment Testing**: Automated testing of software deployment across representative computing environments

Interview Questions:

1. How do you ensure your software can be installed and run across different computing platforms and architectures?

2. What container solutions or portable build systems do you provide for users?
3. How do you handle compatibility with different hardware accelerators and modern computing architectures?
4. What documentation supports users deploying your software in diverse institutional computing environments?
5. How do you test software functionality across different platforms and resolve compatibility issues?

Considerations

Container presence doesn't guarantee cross-platform functionality without proper testing and maintenance. Different research communities have varying infrastructure constraints and technical capabilities. Hardware compatibility requirements vary dramatically across different research applications. Documentation must account for institutional variations in computing environments and security requirements.

4.4.6 Maintainability and Understandability

This metric assesses code complexity, documentation quality, structural characteristics, and development practices that support long-term maintenance and contributor onboarding.

Metric-rationale

High maintainability and understandability are crucial for long-term software sustainability, enabling new developers to comprehend and modify the code easily. Well-structured, documented code reduces onboarding time, decreases the likelihood of bug introduction, and supports knowledge transfer among contributors. Modern static analysis tools provide sophisticated assessment of code complexity and maintainability factors beyond traditional metrics.

Measurement Methods

Repository Mining Approach:

- **Advanced Complexity Analysis**: Modern tools assessing cyclomatic complexity, cognitive complexity, nesting depth, and maintainability indices with AI-powered insights
- **Code Quality Assessment**: Comprehensive analysis of code duplication, design patterns, architectural consistency, and technical debt indicators
- **Documentation Quality Evaluation**: Assessment of code comments, API documentation, architectural documentation, and developer onboarding materials

- **Knowledge Distribution Analysis**: Evaluation of code ownership patterns, contributor knowledge distribution, and bus factor assessment for maintainability risks
- **Refactoring and Evolution Tracking**: Analysis of code evolution patterns, refactoring frequency, and maintenance burden trends over time

Interview Questions:

1. How do you ensure code remains understandable and maintainable for new contributors?
2. What documentation practices, coding standards, and architectural principles guide development?
3. How do you handle technical debt, code complexity, and maintenance burden over time?
4. What onboarding processes help new developers understand and contribute to the codebase effectively?
5. How do you balance code optimization with maintainability and contributor accessibility?

Considerations

Code duplication may be acceptable for performance optimization, dependency management, or architectural reasons requiring contextual evaluation. Complexity metrics should be interpreted in the context of problem domain requirements and performance constraints. Documentation quality assessment requires manual evaluation beyond automated presence detection. Maintainability practices should be balanced with development velocity, resource constraints, and project-specific requirements.

4.4.7 Performance and Efficiency

Performance and Efficiency measures computational efficiency, resource utilization, optimization practices, and environmental impact that support scalability and sustainable computing.

Metric-rationale

Efficient software consumes fewer computational resources, reduces energy consumption, and provides a better user experience while supporting environmental sustainability goals. Performance considerations are particularly important for research software that may run on large-scale systems, process significant datasets, or be used repeatedly across research workflows. The integration of performance optimization with environmental impact assessment supports both computational efficiency and sustainability goals.

Measurement Methods

Repository Mining Approach:

- **Performance Benchmarking Integration**: Automated assessment using established benchmark suites, including HPC Challenge, SPEC benchmarks, and domain-specific performance tests
- **Environmental Impact Assessment**: Integration with ISO/IEC 21031:2024 SCI specification for carbon footprint measurement and energy efficiency optimization
- **Resource Utilization Analysis**: Comprehensive monitoring of CPU efficiency, memory usage patterns, I/O performance, and GPU utilization optimization
- **Scalability Assessment**: Analysis of parallel computing support, distributed system capabilities, and performance scaling characteristics
- **Optimization Practice Evaluation**: Assessment of compiler optimization usage, profiling tool integration, and performance-focused development practices
- **Memory Efficiency Analysis**: Profiling tools (Valgrind, Heaptrack) to measure memory footprint, leak detection, and allocation patterns
- **I/O Performance Profiling**: Darshan, benchmarks for parallel I/O characterization
- **Algorithmic Complexity Assessment**: Static analysis tools to identify computational complexity, with validation through scaling studies
- **Power Measurement Integration**: RAPL (Running Average Power Limit) interface, NVML for GPU power, external power meters for validation
- **Performance Portability Assessment**: Consistent benchmarking across CPU architectures (x86, ARM, POWER), GPU vendors (NVIDIA, AMD, Intel), and accelerators

Interview Questions:

1. What performance requirements and optimization strategies guide your software development?
2. How do you measure, monitor, and improve software performance and resource efficiency over time?
3. How does your software scale on large systems, with large datasets, or across distributed computing environments?
4. What trade-offs do you make between performance optimization, code maintainability, and environmental impact?
5. How do you balance computational efficiency with energy consumption and sustainability considerations?
6. What profiling tools do you use to identify performance bottlenecks, and how often do you profile?
7. How do you ensure your software performs consistently across different hardware architectures?
8. What is your approach to balancing numerical accuracy with computational performance?
9. How do you measure and optimize memory usage, particularly for large-scale problems?

10. What performance regression testing do you implement to catch performance degradation?

Considerations

Static analysis detects local performance issues but may overlook algorithmic complexity problems that require domain expertise to identify. Benchmark suites must be maintained, representative of real-world usage, and appropriate for the software's intended applications. Performance optimization should be balanced with code clarity, maintainability, and environmental impact considerations. Performance requirements vary significantly across research domains, computing environments, and use cases, requiring context-specific evaluation approaches.

4.5 Conclusion and Strategic Recommendations

The integration of recent research findings with the CASS Sustainability Metrics framework provides a comprehensive, evidence-based approach to software sustainability assessment. The convergence of standardized methodologies (ISO SCI, FAIR4RS), automated measurement tools (AI-powered platforms and an enhanced CHAOSS ecosystem), and validated implementation approaches positions the field to transition from theoretical frameworks to evidence-based practice.

Key Strategic Recommendations:

1. **Adopt Standardized Frameworks**: Implement ISO/IEC 21031:2024 SCI and FAIR4RS principles as baseline requirements with automated assessment tools.
2. **Leverage Modern Tool Ecosystem**: Transition to AI-powered analysis platforms with significantly improved accuracy and comprehensive assessment capabilities.
3. **Address Validation Gaps**: Acknowledge empirical limitations and prioritize metrics with demonstrated predictive accuracy through longitudinal validation.

Organizations adopting these improved frameworks will access advanced, automated sustainability assessment tools while also helping build the evidence base necessary for field-wide progress in metrics.

5 Emerging and Supplementary Metrics

5.1 Environmental Efficiency

Environmental Efficiency measures the software's carbon footprint and energy efficiency using the ISO/IEC 21031:2024 Software Carbon Intensity (SCI) specification, providing a standardized assessment of environmental sustainability.

Metric-rationale

As computational research increasingly contributes to global carbon emissions, measuring and optimizing the environmental impact of software has become crucial for long-term sustainability. The ISO/IEC 21031:2024 SCI specification provides the first international standard for software carbon measurement, enabling consistent assessment and improvement of software carbon footprint. Research software, particularly in high-performance computing contexts, can have a substantial environmental footprint that affects both direct sustainability and institutional carbon reduction goals.

Measurement Methods

Repository Mining Approach:

- **SCI Automated Calculation**: Implementation of ISO/IEC 21031:2024 formula: SCI = (E × I + M) per R, where E=energy consumption, I=carbon intensity, M=embodied carbon, R=functional unit
- **Green Software Foundation Integration**: Utilize Impact Framework plugins for automated CI/CD pipeline integration with real-time carbon footprint tracking
- **Energy Profiling Tools**: Integration with tools like Intel VTune, NVIDIA Nsight, and CodeCarbon for comprehensive energy consumption analysis
- **Cloud Carbon Tracking**: API integration with cloud providers' carbon reporting tools (AWS Carbon Footprint Tool, Google Cloud Carbon Footprint, Azure Sustainability Calculator)
- **HPC Center Integration**: Direct measurement integration with facility energy monitoring systems at major computational centers

Interview Questions:

1. Have you measured your software's energy consumption or carbon footprint?
2. What strategies do you use to optimize energy efficiency in your software design?
3. How does your software perform on energy-efficient hardware (ARM processors, modern GPUs)?
4. Do you consider environmental impact when making algorithmic or architectural decisions?
5. How do you balance computational performance with energy efficiency requirements?

Considerations

Environmental efficiency measurement is still emerging, with limited baseline data for comparison across software types. SCI calculations require accurate energy consumption data,

which can be challenging to obtain in shared computing environments. The relationship between code optimization and energy efficiency is complex and varies significantly across different hardware platforms. Many research software projects lack the resources or expertise to implement comprehensive environmental monitoring. Carbon intensity varies dramatically by geographical location and energy grid composition, requiring location-aware calculations.

5.2 HPC Scalability and Accelerator Utilization

HPC scalability and accelerator utilization assess the software's capability to effectively use parallel computing architectures, including multi-node systems and specialized hardware accelerators such as GPUs, to address large-scale problems.

Metric-rationale

Modern scientific and engineering breakthroughs increasingly depend on High-Performance Computing (HPC) systems, which combine thousands of processors and accelerators. For research software, the ability to scale efficiently across multiple nodes and effectively utilize GPUs is critical. Poor scalability or underutilization of accelerators can result in wasted computational resources, prolonged time-to-solution, and significant energy consumption. This metric evaluates the software's design and implementation in relation to large-scale parallel execution, ensuring it can effectively address computationally demanding research challenges.

Measurement Methods

Technical Assessment:

- **Scaling Efficiency Analysis:** Quantitative measurement of **strong scaling** (fixed problem size, increasing processors) and **weak scaling** (problem size per processor is fixed, increasing processors). This reveals how performance changes as more resources are allocated.
- **GPU Profiling and Utilization:** Use of specialized tools (e.g., NVIDIA Nsight, AMD ROCm Profiler) to measure key accelerator metrics:
    - **Kernel Occupancy & Execution:** Analysis of how effectively the GPU's streaming multiprocessors are kept busy.
    - **Memory Bandwidth Utilization:** Assessment of data transfer rates between CPU host memory, GPU device memory (HBM), and across the PCIe bus.
    - **Compute-to-Data-Transfer Ratio:** Evaluation of how much time is spent on computation versus moving data to/from the GPU.
- **Interconnect Performance Analysis:** Profiling of inter-node communication patterns, typically using MPI profiling tools (e.g., Score-P, mpiP), to measure:
    - **Communication Overhead:** The proportion of time spent in communication calls (e.g., MPI_Send, MPI_Bcast) versus computation.

    - **Latency and Bandwidth:** Performance of point-to-point and collective communication operations.
    - **Load Imbalance:** Analysis of wait times across parallel processes, indicating uneven workload distribution.
- **Programming Model Implementation:** Identification of parallel programming models and libraries used, such as **MPI, OpenMP, CUDA, ROCm, OpenACC, SYCL,** or portability layers like **Kokkos**.

Interview Questions:

- What is your parallelization strategy for distributing work across multiple nodes and offloading computation to GPUs?
- How do you characterize the scaling performance of your software? Which scaling model (strong or weak) is more relevant to your target use cases?
- What tools and techniques do you use to identify and resolve performance bottlenecks related to GPU execution or inter-node communication?
- How do you manage data locality and minimize data movement between the host and GPU, as well as across the network interconnect?
- What trade-offs have you made between performance on a specific HPC system and code portability to other architectures?

Considerations

Performance on HPC systems is highly dependent on the underlying hardware, including the specific CPU/GPU models, the network interconnect topology (e.g., InfiniBand, Slingshot), and the parallel file system. Benchmarks must be conducted on representative target hardware for meaningful results. Algorithmic limitations, described by Amdahl's Law, can place a theoretical cap on scalability that is independent of implementation quality. Finally, profiling large-scale, multi-node, multi-GPU applications is a complex task that requires significant expertise and specialized tools to interpret correctly.

6 Conclusion

The CASS Sustainability Metrics framework provides guidance on evaluating and improving research software sustainability. Although validation gaps exist, it integrates best practices and emerging standards into practical recommendations. Success in sustainability assessment depends on balancing thorough evaluations with practical constraints. Projects should see these metrics as tools for strategic planning and ongoing improvement rather than absolute measures of sustainability. As the field evolves, continued validation research will improve the effectiveness of these frameworks. Organizations using these approaches help advance their own software sustainability and support the broader development of evidence-based practices in research software engineering.

7 Bibliography

- Adejumo, E. K., et al. (2025). "An Empirical Validation of Open Source Repository Stability Metrics." arXiv preprint arXiv:2508.01358.
- Avelino, G., et al. (2019). "On the abandonment and survival of open source projects: An empirical investigation." Empirical Software Engineering.
- Barker, M. G., & Katz, D. S. (2022). "The Research Software Landscape and Funding Opportunities." Computer.
- Bessemer Venture Partners (2024). "Measuring the engagement of an open source software community." BVP Atlas. https://www.bvp.com/atlas/measuring-the-engagement-of-an-open-source-software-community
- Boettiger, C. (2015). "An Introduction to Docker for Reproducible Research." ACM SIGOPS Operating Systems Review.
- Carver, J. C., et al. (2013). "Software Engineering for Science." Computing in Science & Engineering.
- CHAOSS Community (2023). "Metrics Model: Starter Project Health." CHAOSS Knowledge Base. https://chaoss.community/kb/metrics-model-starter-project-health/
- CHAOSS Community (2024). "Metric: Contributors." CHAOSS Knowledge Base. https://chaoss.community/kb/metric-contributors/
- CHAOSS Community (2024). "Metric: Issue Label Inclusivity." CHAOSS Knowledge Base. https://chaoss.community/kb/metric-issue-label-inclusivity/
- CHAOSS Community (2024). "Metric: Issue Response Time." CHAOSS Knowledge Base. https://chaoss.community/kb/metric-issue-response-time/
- CHAOSS Community (2024). "Metric: New Contributors." CHAOSS Knowledge Base. https://chaoss.community/kb/metric-new-contributors/
- CHAOSS Community (2024). "Metric: Newcomer Experience." CHAOSS Knowledge Base. https://chaoss.community/kb/metric-newcomer-experience/
- CHAOSS Community (2024). "Metric: Time to First Response." CHAOSS Knowledge Base. https://chaoss.community/kb/metric-time-to-first-response/
- CHAOSS Community (2024). "Metrics Models for Open Source Sustainability."
- CHAOSS Community (2024). "Practitioner Guide - Contributor Sustainability." CHAOSS Knowledge Base. https://chaoss.community/practitioner-guide-contributor-sustainability/
- CHAOSS Diversity, Equity, and Inclusion Working Group (2024). "Psychological Safety Assessment Framework."
- CORSA Center (2024). "Open Source Governance Documents."
- Crowston, K., & Howison, J. (2005). "The Social Structure of Free and Open Source Software Development." First Monday.
- Daily.dev (2025). "Open Source Contributor Onboarding: 10 Tips." https://daily.dev/blog/open-source-contributor-onboarding-10-tips
- Decan, A., Constantinou, E., & Mens, T. (2020). "GAP: Forecasting Commit Activity in git Projects." Journal of Systems and Software, Vol. 165.

- DeepSource (2024). "AI-Powered Code Quality Analysis Platform Technical Documentation."
- Druskat, S., & Katz, D. S. (2021). "Research Software Sustainability and Citation." ICSE Workshop.
- Dunbar-Hester, C., et al. (2024). "Beyond Stars and Forks: Why Open Source Needs Better Collaboration Metrics." DEV Community. https://dev.to/bekahhw/beyond-stars-and-forks-why-open-source-needs-better-collaboration-metrics-hla
- EVERSE Project (2024). "Research Software Quality Toolkit (RSQkit)." European Virtual Institute for Research Software Excellence.
- FAIR for Research Software (2022). "Research Data Alliance / FORCE11."
- GitHub Blog (2023). "Metrics for issues, pull requests, and discussions." The GitHub Blog. https://github.blog/open-source/maintainers/metrics-for-issues-pull-requests-and-discussions/
- GitHub OSPO (2023). "Open Source Health Metrics." GitHub Documentation. https://github.com/github/github-ospo/blob/main/docs/open-source-health-metrics.md
- GitHub ReadME Project (2021). "The secrets to onboarding new open source contributors." GitHub. https://github.com/readme/featured/contributor-onboarding
- GitLab (2025). "How we use GitLab to grow open source communities." GitLab Blog. https://about.gitlab.com/blog/how-we-use-gitlab-to-grow-open-source-communities/
- González-Barahona, J. M., & Robles, G. (2015). "Top 5 open source community metrics to track." Opensource.com. https://opensource.com/business/15/12/top-5-open-source-community-metrics-track
- Green Software Foundation (2024). "Software Carbon Intensity (SCI) Specification." ISO/IEC 21031:2024.
- Gupta, U., Harchol-Balter, M., Schmitz, M., et al. (2020). "Chasing Carbon: The Elusive Environmental Footprint of Computing." Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA).
- Hettrick, S., et al. (2014). "UK Research Software Survey." Software Sustainability Institute.
- Howison, J., et al. (2015). "Software Sustainability and High Energy Physics: Lessons for the Humanities." Preservation of Complex Objects Symposium.
- Katz, D. S., et al. (2018). "Fourth Workshop on Sustainable Software for Science: Practice and Experiences (WSSSPE4) – Workshop Report." Journal of Open Research Software.
- Kaur, R., Chahal, K. K., & Saini, M. (2020). "Understanding community participation and engagement in open source software Projects: A systematic mapping study." Journal of King Saud University - Computer and Information Sciences. https://doi.org/10.1016/j.jksuci.2020.10.020
- Kim, B., et al. (2025). "Analyzing Key Features of Open Source Software Survivability with Random Forest." Applied Sciences, 15(2), 946. https://www.mdpi.com/2076-3417/15/2/946

- Loewe et al. (2021). "Environment for Sustainable Research Software in Germany." F1000Research.
- National Institutes of Health (2024). "Research Software Engineer (R50) Award Program."
- Nyenah, E., et al. (2024). "Software sustainability of global impact models." Geoscientific Model Development.
- Open Source Guides (2025). "Building Welcoming Communities." GitHub. https://opensource.guide/building-community/
- Open Source Guides (2025). "How to Contribute to Open Source." GitHub. https://opensource.guide/how-to-contribute/
- Open Source Guides (2025). "Open Source Metrics." GitHub. https://opensource.guide/metrics/
- Research Software Alliance (2024). "The FAIR for Research Software Principles after two years: an adoption update."
- Research Software Engineering Reports (2019–2021). "Professionalizing Research Software Development: Community Reports and Strategies."
- Runa Capital (2024). "Open Source Contributors Analysis and OS Databases." Medium. https://medium.com/runacapital/open-source-analysis-and-os-databases-1eb1fe840719
- Sandve, G. K., et al. (2013). "Ten Simple Rules for Reproducible Computational Research." PLOS Computational Biology.
- Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2020). "Green AI." Communications of the ACM.
- Sharma, A. (2019). "10 tips for onboarding open source contributors." Opensource.com. https://opensource.com/article/19/12/open-source-contributors
- Smith, A. M., Katz, D. S., Niemeyer, K. E., et al. (2016). "Software Citation Principles." PeerJ Computer Science.
- Software Sustainability Institute (2023). "SSI Midterm Review."
- Steinmacher, I., et al. (2018). "Let Me In: Guidelines for the Successful Onboarding of Newcomers to Open Source Projects." IEEE Software. https://www.researchgate.net/publication/322414294
- Stodden, V., Leisch, F., & Peng, R. D. (2014). "Implementing Reproducible Research." CRC Press.
- TeachingRSE Collaboration (2023). "Foundational Competencies and Responsibilities of a Research Software Engineer."
- Turzo, A. K., Sultana, S., & Bosu, A. (2024). "From First Patch to Long-Term Contributor: Evaluating Onboarding Recommendations for OSS Newcomers." arXiv preprint arXiv:2407.04159. https://arxiv.org/abs/2407.04159
- Venters, C. C., Lau, L., Griffiths, M. K., Holmes, V., Ward, R. R., & Xu, J. (2014). "Software Sustainability: Research and Practice from a Software Engineering Perspective." Journal of Systems and Software.
- VMware Open Source Blog (2022). "When and How to Deprecate an Open Source Project."
- Watson et al. (2023). "Sustainable Research Software Institute (SRSI) Model." arXiv.

- Zhou, M., & Mockus, A. (2010). "Growth of New Projects in GitHub." International Conference on Software Engineering (ICSE).